LitCode

Praneet Kapoor

July 18, 2023

1 Introduction

1.1 Artistic Nature of Computer Programs

The study of computers and computations is a science. However, computer programming is an art. To understand the difference between art and science we can read the following discussion on the subject by John Stuart Mill:

> Several sciences are often necessary to form the groundwork of asingle art. Such is the complication of human affairs, that to en-able one thing to be done, it is often requisite to know the natureand properties of many things. ... Art in general consists of thetruths of Science, arranged in the most convenient order for prac-tice, instead of the order which is the most convenient for thought.Science and its truths so as to enable us to takegroups arrangesin at one view as much as possible of the general order of the uni-verse. Art ... brings together from parts of the field of science mostremote from one another, the truths relating to the production ofthe different and heterogeneous conditions necessary to each effectwhich the exigencies of practical life require[1].

A programmer writes a sequence of instructions to be given the computer so that a desired result can be achived or 'computed by the computer'. This sequence of instructions are what we call a program. He[1] has to skillfully use their knowledge about different subject, then learn some more, and then write the program step by step. There is lots of testing and debugging involved. There is no well defined way to writing a program. Hence, it is an art.

The recent advancements in the field of machine learning have meant that large language models are able to put together a crappy code to do some stuff, it may very well be possible that we do indeed make computer programming a science. However, that would not take away the joy of writing programs. AI systems can be Chess and Go much better than most of the humans on this entire planet, but we still play these games because they are fun. Same goes for painting and photography — we can tell a system like DALL-E to cook up a painting or a drone to take the photograph of forest. However, humans will still continue to paint and click photographs just because doing so gives us pleasure.

[1] Read 'He' as 'He/she' throughout this article

1.2 Structured Programming Methodology and Literate Programming

Structured programming methodology arose as a way to improve clarity, quality and development time of programs. It involves constructing a program using the three basic constructs — sequential statements which occur and are executed one after the other, a conditional statement (`if`/`else`), and iterative statments (`for` and `while` loops). Using these three basic constructs we can build simple blocks and functions which work together to produce the result we desire.

Though structured programming allows us to write programs with some discipline, the act writing a program is still an art. Some later parts of a program might have been written before the parts which are executed at the entry point. Moving from A to B in a program might have involved several complex steps, which a bunch of comments won't explain well. When we see a program, we essentially see a structure which was made out of a *web* of ideas. This is the reason why we, programmers, have trouble understanding certain pieces of code which we had written a few months back. When we wrote them, we were in the *zone* and were able to craft an edifice out of scattered ideas. Once we are done with a code, we are out of that zone. It takes considerable effort to go from the edifice to the *web* of ideas which created it so that we can understand the paths to various rooms in the building.

We can remedy this by documenting our code. Every good programmer knows the utility of a good documentation. However, almost every programmer hates to write documentation for their code. Why? Why would we hate to write something which will only help us and others in understanding something?

A primary reason for this general dislike is the distractive nature of writing a documentation. Even with all its utility, it can become a distraction in a world where releases of a code base containing new features and bug fixes are to be shipped out at a very rapid pace.

Also, a documentation might not end up explaining matters clearly sometimes. After all it is explaining the structure and not the ideas which lead to the creation of the structure. This documentation will be helpful for someone who wishes to use a library in their project, but it won't be of a great help for those who wish to understand the underlying construction of the structure.

To remedy the deficiencies of 'traditional' programming and documentation, Donald Knuth came up with the concept of *Literate Programming*[2]. In literate programming, you don't write structured programs. You essentially write an essay which contains 'chunks' of the source code present throughout it. The prose explains the motivation and working of chunks. The source code inside a chunk can be expanded by referencing other chunks from inside it or by simply expanding its definition in the later part of the essay. We get the full program by combining the chunks together, a process which Knuth calls *tangling* as we are producing a tangled up piece out of web of ideas, something which a computer can understand easily but some person not involved in the development might have trouble understanding. To produce a documentatiom, you pass the essay through a markup processing engine like TEXor LATEXto produce a `DVI` or `PDF` file. This is what Knuth calls *weaving* the web — you weave the web to produce a beautifully typeset documentation which explains your code and its API, both for programmers and for users.

The original literate programming system was `WEB` which was used by Knuth when writing

TEXtypesetting engine. It used a program called `tangle` which took `.web` files and output the source code for TEXengine written in PASCAL. Another program `weave` was used to produce the documentation in `DVI` format. The `WEB` could essentially be thought of as a bunch of macros which were built up on top of TEX. A version of `WEB` for C programs called `CWEB` was developed by Donald Knuth and Silvio Levy and still used to this day by Knuth and others [1].

Sometime later, Norman Ramsey came up with `noweb` literate programming system[3]. It was an improvement over `WEB` and `CWEB` thanks to its simplicity which arose from the simple markup it used to enclose and refer to code chunks. Code chunks were encoded with `«Name of the chunk»=` and `@`. You could refer to another chunk from inside a chunk by enclosing the name of the reference inside double angles. A short example of a literate program written in the `noweb` style is shown in .

In the terminal you will type `notangle source.web -Rhello.c > hello.c` to extract the source code. To produce a LATEXfile, you will execute `noweave` : `noweave source.web -latex > source.tex` .

1.3 Did Literate Programming take off?

In a way, yes. We have got Jupyter Notebooks and Mathematica, which provide *Notebook* interfaces where you write your code like you usually do but intersperse it with chunks of markdown text explaining it. It has its merits — scientific collaboration is one of them, which led to the code written in Jupyter notebooks being used to detect gravitational waves. However, notebooks aren't full literate programming systems as they do not allow you to place chunks as per your wish or refer to them.

Physically Based Rendering , a book on rendering of images in a way that imitates optics in real world, is actually a big literate program which explains the `pbrt` library. Its authors won an Academy award as their work was used in movies to render objects more realistically [4].

In fact, full literate programming systems took off a bit. Which is sad knowing that had this approach to programming taken off completely, then programming would have been made more accessible to people and the quality of code would have improved drastically. Someone on ycombinator said (roughly, I don't remember the exact words), "Literate programming requires discipline. It is not suited for world where software releases are done at such a fast pace." — a statement which I found to be true once I started working in a corporate.

1.4 Why then another literate programming tool?

I could have used `noweb` but I suffer from NIH - 'Not Invented Here' syndrome. The `-L` in `notangle` didn't seem to respect the indentation of reference in parent chunk which is not good for Python code. Its weaved output was largely unreadable for me. And the hacker guide for it was somewhat confusing — I could never figure out how I can change styles or fix the operation of `-L` flag[5].

So, I decided to write LitCode. It consists of four programs:

1. `linit` : Generates `litcode.sty` file used for styling and referencing chunks.
2. `ldump` : Prints JSON containing chunks present in files passed to it as command line arguments.

3. `ltangle` : Using the JSON generated by `ldump` it expands a chunk/reference which has been passed to it as command line argument.

4. `lweave` : Produces LaTeXequivalent of files passed to it as command line arguments. It transpiles the markup characters used by LitCode to their LaTeXcounterparts.

No one is going to probably use this tool except for me. Given my good experiences with literate programming I feel that I will be using this system for my life!! So without any more nonsense, lets write the source code for LitCode.

Note: Some code is present in `bootstrap/` which is used to build the current version of programs making up LitCode system. Those pieces are ran first (as you will see in the `Makefile` in the parent directory), and then the newly built LitCode is used to tangle and weave the `web` files.

2 Source Code

2.1 Styling file

I will begin by defining the styling file which will be used by LaTeXto style the chunks and references. This makes up `litcode` package. I will be honest — I am not that good in TeXand LaTeX. Initially I took lots of help from ChatGPT to cook `litcode.sty` file. Later on, I decided to keep things simple, did some additions and deletions, and ended up with the current version of `litcode.sty` file.

```
⟨litcode.sty⟩ ≡
⟨Packages required by LitCode⟩
⟨Variables in LitCode⟩
⟨hyperref and url style setup⟩
⟨code environment⟩
⟨oldcode environment⟩
⟨vcode environment⟩
⟨Caption and Chunk Listing format⟩
```

After lots of hits and trials I ended up with the following list of packages which will be used by LitCode. A more clever user of LaTeX can maybe add and delete a few packages from this chunk.

```
⟨Packages required by LitCode⟩ ≡
\RequirePackage[T1]{fontenc}
\usepackage{amsmath}
\RequirePackage{textcomp}
%\RequirePackage{inconsolata}
\RequirePackage{calc}
\RequirePackage{caption}
\RequirePackage{hyperref}
\RequirePackage{cleveref}

\RequirePackage{fancyvrb}
\RequirePackage{listings}
\RequirePackage{xcolor}
```

```
\RequirePackage{xparse}
\RequirePackage{subcaption}
```

Let me first tell you about the environments. There are three flavours of environments in LitCode, all of them being derived from `lstlisting` environment:

1. `code` : Used when defining a new chunk. Caption and labels will be set for this environment. Captions will be invisble so that no weird indentations or spaces are there between normal text and listing. `@<` and `@>` are used to escape inside this environment so that we can refer to other chunks.

2. `oldcode` : Same as `code` but used when extending the definition of a previously defined chunk. No caption and label is defined for this environment.

3. `vcode` : Used for writing a purely verbatim piece of code. No captions, no labels, no escape characters.

For these environments, I have defined a few variables which will allow us to set listing font, background color, margin and textwidth, although the latter two aren't used at all (they are just relics of a once complicated `.sty` file that I had).

```
⟨Variables in LitCode⟩ ≡
\def\litcode{{LitCode}}
\def\linewidth{\textwidth}

\def\xleftmargin{10pt}
\def\xrightmargin{0pt}

\def\litcodefont{\fontfamily{SourceCodePro-TLF}\small}
\def\codebg{yellow!15}
\def\vcodebg{green!20}
```

I also wanted to enable 'jumps' whenever I would click on reference + I wanted these references to be colored differently. Thats why I have set `colorlinks` as `true` and have set both `linkcolor` and `\urlcolor` to `red` .

```
⟨hyperref and url style setup⟩ ≡
\hypersetup{
    colorlinks=true,
    linkcolor=red,
    urlcolor=red,
    citecolor=red
}
```

Thanks to the line below, case you are using a URL, it will be put in the same style as the surrounding text.

```
⟨hyperref and url style setup⟩ +≡
\urlstyle{same}
```

Now come the interesting chunks, which troubled me a lot — definition of custom environments, caption and chunk listing style. I will begin with the queen — the `code` environment.

```
⟨code environment⟩ ≡
% Define code environment: Used when defining a chunk for the first time
\lstnewenvironment{code}[1][]{
    \lstset{
```

⟨*code environment*⟩ takes in one argument which is used to set both the caption and label of the environment. Note how I am using `\protect\detokenize{}` to make sure the formatting is not messed up. Do note that captions will be invisible so that we can have captions and code chunk indented by the same level. Also 'fake captions' will be inserted to by `lweave` with proper formatting so that chunk definitions and extensions can be distinguished from each other.

```
⟨code environment⟩ +≡
        caption={\protect\detokenize{#1}},
        label={#1},
```

We further extend ⟨*code environment*⟩ by setting `columns` to `fullflexible` so that the letters are nicely typeset (unlike in `fixed` column width). I have set the font to `\ttfamily` — you can change it to something else. `keepspaces` , and `breaklines` are used to make sure the indentation is not changed and the lines are wrapped if they go over margins, respectively. `upquote = true` ensure that double quotes are formatted as simple upquotes only (and not as " and "). `background` is used to set the background color via `\codebg` variable: set it to any color you wish in the preamble.

```
⟨code environment⟩ +≡
        columns=fullflexible,
        basicstyle=\litcodefont,
        keepspaces=true,
        breaklines=true,
        upquote=true,
        backgroundcolor=\color{\codebg},
```

The interesting part is the `escapeinside` variable, which is used to set the characters which will be used to escape inside the `lstlisting` environment. I have set these variables to `@<` and `@>`. To print these variables inside the environment, they need to be escaped themselves. Example:

«»= @<@<@> @<@>@> @

```
⟨code environment⟩ +≡
@<@>
    }
}{}
```

We can define `oldcode` and `vcode` environments in a similar fashion with a few minor changes: in `oldcode` we will get rid of caption and label, and in `vcode` we will get rid of escape characters.

```
⟨oldcode environment⟩ ≡
% Define oldcode environment: To be used when extending an already defined
    chunk
\lstnewenvironment{oldcode}[1][]{
    \lstset{
        columns=fullflexible,
        basicstyle=\litcodefont,
        keepspaces=true,
```

```
        breaklines=true,
        upquote=true,
        backgroundcolor=\color{\codebg},
@<@>
    }
}{}
```

```
⟨vcode environment⟩ ≡
% Verbatim code environment: Takes no captions, has no labels and won't get
    included in the listing.
% Purely verbatim!!
\lstnewenvironment{vcode}[1][]{
    \lstset{
        columns=fullflexible,
        basicstyle=\litcodefont,
        keepspaces=true,
        breaklines=true,
        upquote=true,
        backgroundcolor=\color{\vcodebg},
    }
}{}
```

We end this `.sty` file by defining the format in which we want to display captions in references and in listings. I have followed the style which had been used by Knuth and Ramsey:

- ⟨ name of the chunk ⟩ ≡: Used when defining a chunk for the first time
- ⟨ name of the chunk ⟩ +≡: Used when extending the definition of a chunk
- ⟨ name of the chunk ⟩: Used when referencing a chunk

The first and third type of formatting is defined in `litcode.sty` itself. One can cook up a LaTeXcode which checks whether a chunk has been previously defined or not and accordingly format its captioning. I am not that clever. This decision (checking the *freshness* of the chunk and formatting its caption) is done by `lweave` .

```
⟨Caption and Chunk Listing format⟩ ≡
\DeclareCaptionFormat{codecaptionformat}
{
    \phantom{$\langle\textit{#3}\rangle\!\!\equiv$}
}

\captionsetup[lstlisting]{
    format=codecaptionformat,
    justification=raggedright,
    singlelinecheck=off,
    skip=-\baselineskip
}

\makeatletter
\renewcommand\l@lstlisting[2]{\@dottedtocline{1}{0em}{2.3em}{\textit{#1}}{#2}}
```

```
\makeatother

\newcommand{\coderef}[1]{$\langle$\textit{\nameref{#1}}$\rangle$}
%\newcommand{\coderef}[1]{$\langle$\textit{\nameref{#1}} \rm(p.~\pageref{#1})$\
    rangle$}
```

Note how in `\DeclareCaptionFormat` I have set the caption under `\phantom` command so that they are invisble and do not mess up the indentation. In `\captionsetup` I have setup `skip` to `-\baselineskip` so that the vertical space between a chunk and lines of normal text preceding it is not huge.

2.2 linit.py

`linit.py` or simply `linit` , will be responsible for writing `litcode.sty` to a directory which is a passed to it as a command line argument. Of course of course, I can upload this styling file CTAN, but I do not think that this is anything ground-breaking, so I will keep it with myself.

The first line of `linit.py` and all other Python scripts will begin with a shebang (`#!`) followed by the path to the binary of python3 interpreter. All these scripts will be set as executables (`chmod +x`) and then moved to `/local/bin/` so that we can run them from anywhere in the system without explicitely calling Python to execute them everytime.

```
⟨linit.py⟩ ≡
#!/usr/bin/python3
import os
import sys
import time
import subprocess as sbpr
```

The entire content of `litcode.sty` will be present inside `linit.py` as a multi-line raw string. Why a raw string? This is because this `.sty` file makes extensive use of backslash \. I do not want to make things messy by escaping every backslash... Better to just use a raw string. :-)

```
⟨linit.py⟩ +≡
litcode_sty_content = r'''
⟨litcode.sty⟩
'''
```

`linit.py` will accept only a single command line argument which would be the name of the directory where `litcode.sty` is to be saved. The 'entry-point' for our script will be the `main` function, where the command line arguments will be processed to get this directory name. If none is supplied, then the program should exit with an error code.

```
⟨linit.py⟩ +≡
def main():
    global litcode_sty_content
    if len(sys.argv) <= 1:
        print('Please supply a directory name')
        exit(1)
    directory = sys.argv[1]
```

We can also ensure that the directory is made using `mkdir -p` . It won't do any harm to the directory's contents if it already exists.

```
⟨linit.py⟩ +≡
    command = f'mkdir -p {directory}'
    os.system(command)
```

Finally, we write the raw string to `litcode.sty` placed under the directory supplied as a command line argument. We need the full path to `litcode.sty` which can easily be made by appending the name of the directory (relative to the current directory/position in the terminal), system path separator (which we get by `os.sep()`) and the string `'litcode.sty'`. I have just used a formatted string, although these strings could also have been appended by `+` operator.

```
⟨linit.py⟩ +≡
    full_path_to_sty = f'{directory}{os.sep}litcode.sty'
    with open(full_path_to_sty, 'w') as fp:
        fp.writelines(litcode_sty_content)

main()
```

2.3 ldump.py

Tangling a chunk involves recursive expansion of all the references that are there in the chunk. The indentation of a reference must be appended while printing all the lines of the chunk it refers to. We also need to add some markers which tell a programmer from where a piece of code lies in the WEB files. We also need to ensure that proper whitespace conversion is done — this is especially useful in the case of `Makefile` which accepts only tabs as valid indentation characters.

Most of the traditional literate programming tools tangle a chunk by going through a file, making a dictionary of chunks and then expanding the required chunk recursively. What would happen if we need to tangle 2 chunks from the same file. Some computational power will be expended in setting up the dictionary twice. To prevent this 'waste of computational power' (although in the 21st century, hardly anyone cares about it for such simple text processing job), I have written `ldump.py` . It will make a dicitionary of chunks which are present in the files which have been supplied to it as command line arguments. Then this dictionary will be printed and can be saved in a JSON file using the clobber operator, `>` . This JSON will be used by `ltangle` when tangling a piece of code.

NOTE: If a chunk is defined in multiple files and have no relation to each other, give them different names. This will prevent `ltangle` and `lweave` from considering them as related to each other.

We begin with the shebang line specifying the path of the python3 and import `os`, `sys`, `json` and `re` modules.

```
⟨ldump.py⟩ ≡
#!/usr/bin/python3
import os
import sys
```

```
import re
import json
```

A pseudocode for `ldump.py` is presented in Algorithm 1:

Algorithm 1: Pseudocode for `ldump.py`

1. Check if command line args have been supplied.
   If yes: then start reading each file line by line
   If not: then throw an error
2. Go through the string and look lines satisfying the regex:
   `(?<=^\coderef{).+?(?=}=\s*$)`
3. Check if the chunk exists or not. If it is not their in the dictionary, then enter it. Else, we will append the lines in this chunk to the already existing lines in the dictionary.
4. Add some metadata about the position of chunk boundaries in the web files.
5. JSONify the dictionary and print it. Writing to the file will be taken care of by `>` operator.

We will be having a function `dumper` which will take in a list of files, read them line by line and create a dictionary of chunks, `chunkDict` . This dictionary will then be converted into a pretty-printed JSON string using `dumps` method in `json` package.

```
⟨ldump.py⟩ +≡
def dumper(file_list):
    chunkDict = dict()
    for f in files:
        ⟨Fill chunkDict⟩
    chunkJSON = json.dumps(chunkDict, indent = 4)
    return chunkJSON
```

Our entry point into the program will be `main()` which will see if the files have been supplied as command line args or not. If they exist, we will send this list to the `dumper` . The JSON string that it gets from the `dumper` will be printed on the screen.

```
⟨ldump.py⟩ +≡
def main():
    if len(sys.argv) <= 1:
        print('No files provided')
        exit(1)
    file_list = sys.argv[1:]
    json_str = dumper(file_list)
    print(json_str)

main()
```

We now have to write some code which will write `chunkDict` . When reading a file, we need to keep track of the following:

1. Line number of the current line being read.
2. Are we in a chunk or not?
   - If not, then does this line defines a chunk? Is it really new or an old one?

- If yes, then does this line refers to another chunk? Is there a `@` in this line? (which marks the end of a chunk's definition)

We keep track of these conditions with the help of `line_count`, `inChunk`, `encounteredReference`, and `encounteredStopChar`variables.

```
⟨Fill chunkDict⟩ ≡
line_count = 0
inChunk = False
encounteredReference = False
encounteredStopChar = False
```

We will read all the lines of a file and store them in a list (`lines`) in one shot using `readlines` method. `chunkLines` will store the lines of a chunk as and when we encounter. It will be off-loaded into `chunkDict` .

```
⟨Fill chunkDict⟩ +≡
with open(f, 'r') as fp:
    lines = fp.readlines()
chunkLines = []
```

Using a `for` loop I am now going to iterate over `lines` . The first step in each iteration would be to increment `line_count`.

```
⟨Fill chunkDict⟩ +≡
for line in lines:
    line_count += 1
    ⟨Fill chunkDict:  Not in a chunk⟩
    ⟨Fill chunkDict:  In a chunk⟩
```

There will be two conditions. Either I will be in a chunk or I won't. If I am not in a chunk, `inChunk` is set to `False` , and I have to check whether the current line is the beginning of a chunk definition or not.

```
⟨Fill chunkDict:  Not in a chunk⟩ ≡
if not inChunk:
    chunkname = re.findall(r'(?<=^⟨??⟩=\s*$)', line.strip())
    if chunkname:
        inChunk = True
```

I now need to check whether the chunk has been previously defined or not. This can be easily done with the use of `in` operator. Remember that `re.findall()` returns a list, so I need to set `chunkname` to the first element of the list that has been returned by `findall()`.

```
⟨Fill chunkDict:  Not in a chunk⟩ +≡
    chunkname = chunkname[0]
    if chunkname not in chunkDict:
        chunkDict[chunkname] = []
        chunkLines = []
        chunkLines.append(f'@->{chunkname}: Starts at line {line_count} in {f}\
    n')
    else:
        chunkLines = chunkDict[chunkname]
```

```
        chunkLines.append(f'@->{chunkname}: Extended at line {line_count} in {f
    }\n')
```

Note how I am introducing metadata beginning with `@->` at the beginning of the chunk.

I have to now deal with the condition in which I am inside a chunk. When inside a chunk, I need to check whether the current line contains a reference or a stop-character (signaling end of the chunk), and accordingly insert some metadata.

```
⟨Fill chunkDict:  In a chunk⟩ ≡
else:
    encounteredStopChar = (line.strip() == '@')
    if encounteredStopChar:
        chunkLines.append(f'@->{chunkname}: Ends at {line_count} in {f}\n')
        chunkDict[chunkname] = chunkLines
        inChunk = False
        continue
    else:
        encounteredReference = re.findall(r'(?<=^⟨??⟩\s*$)', line.strip())
        if encounteredReference:
            chunkLines.append(f'@->{chunkname}: Reference at line {line_count}
    in {f}\n')
            chunkLines.append(line)
            chunkLines.append(f'@->{chunkname}: Continues from line {line_count
     + 1} in {f}\n')
        else:
            chunkLines.append(line)
```

3 Fibonacci Series

The C code presented in this section is responsible for printing the fibonacci series.

I begin by writing the `Makefile` for building the program.

```
⟨Makefile⟩ ≡
CC := gcc -std=C99 -O0

runfib: fibonacci.exe
    ./fibonacci.exe 30

fibonacci.exe: fibonacci.o
    $(CC) fibonacci.o -o fibonacci.exe

fibonacci.o: fibonacci.c
    $(CC) -c fibonacci.c -o fibonacci.o
```

Now comes the fun part. Our code will have two parts: ⟨*Initialize variables*⟩ will be responsible for initializing the variables which will then be used by ⟨*Generate Fibonacci series till n terms*⟩ to generate the series till some n number of terms. n will be provided by the user as a command line argument.

```
⟨fibonacci.c⟩ ≡
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv)
{
    ⟨Initialize variables⟩
    ⟨Generate Fibonacci series till n terms⟩
    fprintf(stdout, "Done\n");
    return 0;
}
```

To generate fibonacci series, we need three numbers: a, b, and n. a and b will be used to generate the terms of the series till n terms. n will be a command line argument. We will need to check whether n has been passed or not.

```
⟨Initialize variables⟩ ≡
int a, b, n;
a = -1;
b = 1;
⟨Check cmd args and Initialize n⟩
```

```
⟨Check cmd args and Initialize n⟩ ≡
if (argc < 2) {
    fprintf(stderr, "Please provide `n' as a command line argument\n");
    exit(1);
}
else {
    n = atoi(argv[1]);
}
```

We also need to make sure that n is not less than zero. So, we will extend the definition of «Check cmd args and Initialize n».

```
⟨Check cmd args and Initialize n⟩ +≡
if (n < 0) {
    fprintf(stderr, "n >= 0\n");
    exit(1);
}
```

Finally, we write the *generating* portion of our code: ⟨*Generate Fibonacci series till n terms*⟩.

```
⟨Generate Fibonacci series till n terms⟩ ≡
for (int i = 0; i < n; ++i) {
    fprintf(stdout, "%d ", a + b);
    b = b + a;
    a = b - a;
}
```

4 Printing Prime Numbers

In this example we are going to print prime numbers. Like in the previous example, we are going to begin by first define the rules by which `prime.c` is to be built.

```
⟨Makefile⟩ +≡
CC := gcc -std=c99 -O0

runprimes: primes.exe
    ./primes.exe 30

primes.exe: primes.o
    $(CC) primes.o -o primes.exe

prime.o: prime.c
    $(CC) -c primes.c -o primes.o
```

Now comes the interesting part: the C code which will print n prime numbers. The core structure of our code will be as follows:

```
⟨primes.c⟩ ≡
#include <stdio.h>
#include <stdlib.h>

int isPrime(int);

int main(int argc, char **argv)
{
    ⟨Initialize n⟩
    ⟨Print prime numbers till n reaches 0⟩
    return 0;
}
```

Like in the previou example, n will be supplied as a command line argument.

```
⟨Initialize n⟩ ≡
int n;
if (argc < 1) {
    fprintf(stderr, "n must be supplied as a command line argument\n");
    exit(1);
}
n = atoi(argv[1]);
```

We will have a loop in which n is decremented by 1 every time a prime number is encountered. Once it reaches 0, the loop ends.

```
⟨Print prime numbers till n reaches 0⟩ ≡
int i = 0;
while (n > 0) {
    if (isPrime(i) == 0) {
        fprintf(stdout, "%d ", i);
        --n;
```

```
    }
    ++i;
}
fprintf(stdout, "Done\n");
```

In the loop, we are calling `isPrime()` to check whether the argument we have passed to it is a prime number or not. This is done by finding the number of divisors of the argument from 2 till $\frac{\text{argument}}{2}$. If the number of divisors is 0, then the argument is a prime number, else not. The edge cases are for 0 and 1, which can be dealt with a simple `if` conditional statement.

```
⟨Check if i is prime or not⟩ ≡
int isPrime(int i)
{
    if (i == 0 || i == 1) {
        return 1;
    }
    for (int d = 2; d <= (i / 2); ++d) {
        if (i % d == 0) {
            return 1;
        }
    }
    return 0;
}
```

And we include it below `main()` in ⟨*primes.c*⟩.

```
⟨primes.c⟩ +≡
⟨Check if i is prime or not⟩
```

Listings

1 litcode.sty 4
2 Packages required by LitCode 4
3 Variables in LitCode 5
4 hyperref and url style setup 5
5 code environment 6
6 oldcode environment 6
7 vcode environment 7
8 Caption and Chunk Listing format 7
9 linit.py 8
10 ldump.py 9
11 Fill chunkDict 11
12 Fill chunkDict: Not in a chunk 11
13 Fill chunkDict: In a chunk 12
14 Makefile 12
15 fibonacci.c 13
16 Initialize variables 13
17 Check cmd args and Initialize n 13
18 Generate Fibonacci series till n terms 13
19 primes.c 14
20 Initialize n 14
21 Print prime numbers till n reaches 0 14
22 Check if i is prime or not 15

References

[1] D. E. Knuth, "Literate programming, volume 27 of," *CSLI Lecture Notes*, 1992.

[2] D. E. Knuth, "Literate programming," *The computer journal*, vol. 27, no. 2, pp. 97–111, 1984.

[3] N. Ramsey, "Literate programming simplified," *IEEE software*, vol. 11, no. 5, pp. 97–105, 1994.

[4] M. Pharr, W. Jakob, and G. Humphreys, *Physically based rendering: From theory to implementation*. MIT Press, 2023.

[5] N. Ramsey, "The noweb hacker's guide," *Princeton University*, vol. 9, 1992.